Laporan Praktikum Dasar Pemrograman **Jobsheet 7**
Nama : Annindyo ayom prakoso
Kelas : TI-1H
No. Presensi: 06

Percobaan 1

1. Komponen Perulangan FOR

Pada kode program Percobaan 1, struktur perulangan for dituliskan sebagai berikut:

Java
```
for (int i = 1; i <= 10; i++) {
    // blok kode perulangan
}
```

Komponen-komponen penyusunnya meliputi:

- **Inisialisasi (Initialization):** int i = 1 — Berfungsi untuk mendeklarasikan dan menginisialisasi variabel penghitung (*counter*) i dengan nilai awal 1.
- **Kondisi (Condition):** i <= 10 — Berfungsi sebagai syarat berjalannya perulangan. Perulangan akan terus dikerjakan selama nilai i bernilai kurang dari atau sama dengan 10.
- **Update (Increment/Decrement):** i++ — Berfungsi untuk menaikkan (increment) nilai variabel penghitung i sebesar 1 setiap kali satu iterasi/putaran selesai dijalankan.

2. Alasan Inisialisasi dan Dampaknya jika Dibalik

- Mengapa diinisialisasi tertinggi = 0 dan terendah = 100?
    - tertinggi diinisialisasi dengan angka terkecil/0 agar saat dibandingkan dengan nilai mahasiswa pertama (yang umumnya bernilai positif ≥ 0), nilai mahasiswa tersebut pasti lebih besar, sehingga variabel tertinggi dapat langsung diperbarui dengan benar.
    - terendah diinisialisasi dengan angka maksimal/100 agar saat dibandingkan dengan nilai mahasiswa pertama, nilai tersebut pasti lebih kecil, sehingga variabel terendah dapat langsung diperbarui dengan benar.

- Apa yang terjadi jika dibalik (tertinggi = 100 dan terendah = 0)?
    - Jika tertinggi diinisialisasi 100 padahal nilai tertinggi dari seluruh mahasiswa yang diinputkan di bawah 100 (misalnya hanya sampai 95), maka variabel tertinggi tidak akan pernah terganti dan program akan tetap menampilkan angka 100 (hasilnya menjadi salah).
    - Sebaliknya, jika terendah diinisialisasi 0 padahal seluruh nilai mahasiswa di atas 0 (misalnya paling rendah adalah 50), maka variabel terendah tidak akan terganti dan akan tetap bernilai 0 (hasilnya juga menjadi salah).

3.Fungsi dan Alur Kerja Potongan Kode Seleksi

Potongan kode:

Java
```
if (nilai > tertinggi) {
    tertinggi = nilai;
}
if (nilai < terendah) {
    terendah = nilai;
}
```

- **Fungsi:** Untuk membandingkan nilai mahasiswa yang baru saja dimasukkan dengan nilai yang saat ini tersimpan di dalam variabel `tertinggi` dan `terendah`, serta memperbaruinya jika ditemukan nilai yang lebih ekstrem (lebih tinggi atau lebih rendah).
- **Alur Kerja:**
    1. Blok `if (nilai > tertinggi)` mengecek apakah nilai input baru lebih besar dari `tertinggi` saat ini. Jika **Ya (true)**, nilai `tertinggi` diperbarui dengan nilai input baru tersebut. Jika **Tidak (false)**, pengecekan dilewati.
    2. Blok `if (nilai < terendah)` mengecek apakah nilai input baru lebih kecil dari `terendah` saat ini. Jika **Ya (true)**, nilai `terendah` diperbarui dengan nilai input baru tersebut. Jika **Tidak (false)**, pengecekan dilewati.

4.



```java
import java.util.Scanner;

public class SiakadFor06 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double nilai, tertinggi = 0, terendah = 100;
        int JMLlulus = 0 , JMLTDKlulus = 0 ;

        for (int i = 1; i <= 10; i++) {
            System.out.print("Masukkan nilai mahasiswa ke-" + i + " :");
            nilai = sc.nextDouble();
            if  (nilai > tertinggi) {
                tertinggi = nilai;
            }
            if (nilai < terendah){
                terendah = nilai;
            }

            if (nilai >= 60) {
                JMLlulus++;
            } else {
                JMLTDKlulus++;
            }
        }

        System.out.println("Nilai tertinggi :" + tertinggi);
        System.out.println("Nilai terendah :" + terendah);
        System.out.println("Jumlah mahasiswa lulus :" + JMLlulus);
        System.out.println("Jumlah mahasiswa tdk lulus :" + JMLTDKlulus);

        sc.close();
    }
}
```

Percobaan 2

1. Maksud dan Kegunaan Sintaks

Java

```
if (nilai < 0 || nilai > 100) {

    System.out.println("Nilai tidak valid. Masukkan lagi nilai yang valid!");
```

```
    continue;
}
```

- **a. nilai < 0 || nilai > 100**
  - **Maksud:** Kondisi logis yang mengecek apakah variabel nilai bernilai kurang dari 0 **atau** bernilai lebih dari 100.
  - **Kegunaan:** Sebagai validasi batas input. Jika pengguna memasukkan nilai di luar rentang 0–100 (misalnya -15 atau 101), kondisi ini bernilai true untuk mendeteksi bahwa nilai tersebut tidak valid.
- **b. continue**
  - **Maksud:** Perintah untuk melompati (*skip*) sisa pernyataan di dalam blok perulangan pada iterasi saat ini dan langsung melanjutkan ke iterasi perulangan berikutnya.
  - **Kegunaan:** Memastikan bahwa nilai yang tidak valid tidak diproses lebih lanjut (tidak dikategorikan ke nilai huruf) dan perulangan while diulang kembali untuk meminta input ulang tanpa menambah *counter* i.

2. Posisi i++ pada Perulangan WHILE

- **Mengapa i++ dituliskan di akhir perulangan WHILE?**
  1. Sintaks i++ ditempatkan di akhir agar penghitung jumlah data valid (i) hanya bertambah **setelah** satu data mahasiswa berhasil diproses dan dikategorikan nilainya secara valid. Jika input tidak valid, perintah continue akan melompati i++, sehingga i tidak bertambah dan perulangan diulang sampai input valid dimasukkan.
- **Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?**
  1. Pengindeksan mahasiswa akan bergeser (misalnya i langsung menjadi 1 pada putaran pertama jika diawali dari 0), sehingga saat mencetak (i + 1) di dalam loop akan menghasilkan indeks yang tidak sesuai.
  2. Apabila terjadi masukan tidak valid dan perintah continue dijalankan, nilai i sudah terlanjur bertambah sebelum input dinyatakan valid, sehingga jatah input data mahasiswa akan berkurang/terlewat.

3. Jumlah Eksekusi Perulangan WHILE

Apabila jumlah mahasiswa (jml) yang dimasukkan adalah **19**:

- Perulangan while (i < jml) diawali dari i = 0 hingga i < 19.
- Perulangan akan berjalan sebanyak **19 kali** (untuk nilai input yang valid).
- *Catatan:* Jika pengguna sempat memasukkan input yang tidak valid, perulangan (proses input) akan mengeksekusi iterasi tambahan sebanyak jumlah input yang invalid tersebut karena diabaikan oleh continue.

4

```java
import java.util.Scanner;

public class SiakadWhile06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int nilai, jml, i = 0;

        System.out.print("Masukkan jumlah mahasiswa :");
        jml = sc.nextInt();

        while (i < jml) {
            System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ":");
            nilai = sc.nextInt();

            if (nilai < 0 || nilai > 100) {
               System.out.println("Masukkan nilai tidak valid. Masukkan lagi nilai yang valid");
               continue;
            }

            if (nilai > 80 && nilai <= 100){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah A");
                System.out.println("Bagus, pertahankan nilainya!");
            } else if (nilai > 73 && nilai <= 80){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah B+");
            } else if (nilai > 65 && nilai <= 73){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah B");
            } else if (nilai > 60 && nilai <= 65){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah C+");
            } else if (nilai > 50 && nilai <= 60){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah C");
            } else if (nilai > 39 && nilai <= 50){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah D");
            } else {
                 System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah E");
            }
            i++;
        }
    }
}
```

Percobaan 3

1.Apabila nama pelanggan yang dimasukkan pertama kali adalah "batal", maka berapa kali perulangan dilakukan?

- Perulangan dilakukan sebanyak **1 kali**. Perulangan langsung berhenti (*break*) pada iterasi pertama tersebut setelah mendeteksi input "batal".

2.Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!

- Kondisi berhenti menggunakan perintah **break** di dalam blok pemilihan if (namaPelanggan.equalsIgnoreCase("batal")).

3.Apa fungsi dari penggunaan nilai true pada kondisi DO-WHILE?

- Nilai true digunakan untuk membuat perulangan berjalan secara **terus-menerus (infinite loop)** secara fleksibel, di mana jalannya program baru akan dihentikan dari dalam blok perulangan menggunakan perintah break saat pengguna mengetikkan kata "batal".

4.Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?

- Karena struktur perulangan do-while dengan kondisi true tidak bergantung pada variabel penghitung (*counter*) seperti pada for atau while, melainkan bergantung pada **kejadian interaktif (aksi/kondisi eksternal)** dari masukan pengguna melalui keyboard hingga pengguna memutuskan untuk mengetikkan "batal".

Tugas

```java
import java.util.Scanner;

public class Bioskop06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int HARGA_TIKET = 50000, totalTiketTerjual = 0, jumlahTiket;
        double totalPenjualan = 0;

        System.out.println("(Masukkan angka 0 untuk mengakhiri transaksi)\n");

        while (true) {
            System.out.print("Masukkan jumlah tiket yang dibeli: ");
            jumlahTiket = sc.nextInt();


            if (jumlahTiket == 0) {
                break;
            }


            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid! Silakan masukkan angka yang benar.\n");
                continue;
            }

            double diskon = 0;
            if (jumlahTiket > 10) {
                diskon = 0.15;
            } else if (jumlahTiket > 4) {
                diskon = 0.10;
            }

            double totalHargaSebelumDiskon = jumlahTiket * HARGA_TIKET;
            double nominalDiskon = totalHargaSebelumDiskon * diskon;
            double totalHargaBayar = totalHargaSebelumDiskon - nominalDiskon;


            totalTiketTerjual += jumlahTiket;
            totalPenjualan += totalHargaBayar;

            System.out.println("Diskon didapat: " + (int)(diskon * 100) + "%");
            System.out.println("Total bayar transaksi ini: Rp " + (long) totalHargaBayar + "\n");
        }

        System.out.println("Total tiket terjual  : " + totalTiketTerjual + " tiket");
        System.out.println("Total pendapatan hari ini : Rp " + (long) totalPenjualan);

        sc.close();
    }
}
```

```java
import java.util.Scanner;

public class Parkir06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int jenis, durasi, total = 0;

        do {
            System.out.print("Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): ");
            jenis = sc.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = sc.nextInt();

                if (durasi > 5) {
                    total += 12000;
                } else {
                    if (jenis == 1) {
                        total += durasi * 3000;
                    } else if (jenis == 2) {
                        total += durasi * 2000;
                    }
                }
                System.out.println("Subtotal biaya parkir saat ini: Rp " + total + "\n");
            } else if (jenis != 0) {
                System.out.println("Jenis kendaraan tidak valid! Masukkan 1, 2, atau 0.\n");
            }

        } while (jenis != 0);

        System.out.println("Total pembayaran parkir: Rp " + total);
        sc.close();
    }
}
```